BATCH — подсистема пакетного исполнения

Руководство программиста

BATCH-PG-0101-RUS

BATCH Version 2.2.6 *** 29-JUL-2026

Содержание

Предисловие **3**
Кому адресовано . . . 3
Состав руководства . . . 4
Обозначения . . . 5

Глава 1. Замысел **6**
1.1 Чем является подсистема . . . 6
1.2 Принятые решения и их причины . . . 7
1.3 Чего намеренно не делается . . . 8

Глава 2. Устройство **9**
2.1 Два образа . . . 9
2.2 Очередь . . . 10
2.3 Путь запроса . . . 11
2.4 Основной цикл . . . 12

Глава 3. Модули **13**
3.1 Карта . . . 13
3.2 batchdef.h . . . 14
3.3 batchmsg.h и batchmsg.c . . . 15
3.4 batchipc.h . . . 16
3.5 util.c . . . 17
3.6 batchd.c . . . 18
3.7 batch.c . . . 19
3.8 Вендоренный подпроект . . . 20

Глава 4. Структуры данных **21**
4.1 BATCH$JOB . . . 21
4.2 BATCHD$JOBENT и BATCHD$CONN . . . 22
4.3 Протокол . . . 23
4.4 Журнал очереди . . . 24

Глава 5. Соглашения **25**
5.1 Имена . . . 26
5.2 Оформление . . . 27
5.3 Соглашение о вызовах . . . 28
5.4 Диагностика . . . 29
5.5 Форматирование вывода . . . 30

Глава 6. Внесение изменений **31**
6.1 Добавление квалификатора . . . 31
6.2 Добавление условия . . . 32
6.3 Добавление команды . . . 33
6.4 Изменение дескриптора задания . . . 34
6.5 Добавление проверки в тест . . . 35

Глава 7. Система сборки **36**
7.1 Состав 36
7.2 Ключи трансляции 37
7.3 Одна неловкость 38

Глава 8. Тесты **39**
8.1 Что есть 39
8.2 Что стережёт основной тест 40

Приложение A. Известные ограничения **41**

Предисловие

Кому адресовано

Руководство предназначено программисту, которому предстоит изменять подсистему: добавить квалификатор, сообщение или команду, устранить дефект либо убедиться, что задуманная правка безопасна. Предполагается знание Си, POSIX и умение читать CMake. Знание OpenVMS не предполагается, хотя тот, кто с ней знаком, поймёт, отчего многое здесь выглядит именно так.

Руководство пользователя описывает, что подсистема делает. Здесь описано, как она устроена, и одно другого не повторяет.

Состав руководства

Часть	Содержание
Глава 1	Чем является подсистема и на каких решениях она стоит
Глава 2	Устройство: два образа, очередь, путь запроса
Глава 3	Модули по одному и назначение каждой рутины
Глава 4	Структуры данных и два формата, переживающих процесс
Глава 5	Соглашения, которых держатся исходные тексты, и их причины
Глава 6	Как вносить изменения, встречающиеся чаще всего
Глава 7	Система сборки
Глава 8	Тесты и то, что они стерегут
Приложение А	Известные ограничения и где каждое из них живёт

Обозначения

Номера строк не приводятся: они устаревают. Вместо этого называется рутина, а каждая рутина несёт в своей шапке блок `FUNCTIONAL DESCRIPTION`, где сказано, что она делает, что принимает и что возвращает.

Глава 1. Замысел

1.1 Чем является подсистема

BATCH выполняет сценарии оболочки как пакетные задания: сценарий помещается в очередь и выполняется позднее, без терминала, под управлением постоянно работающего контроллера заданий. Это функциональный аналог очереди пакетных заданий OpenVMS — `SUBMIT`, `SHOW QUEUE`, `DELETE /ENTRY` — и следует намеренно этому образцу, а не образцу `cron` или супервизора служб.

Из образца вытекают три свойства, которыми определено всё остальное.

- **Задание опознаётся прежде, чем существует.** Подача назначает номер записи очереди и возвращается; процесс порождается позднее, если и когда контроллер так решит. Пользователь держится за запись, а не за процесс.
- **Очередь имеет состояния.** Задание может ждать свободного места либо назначенного времени. Когда ему выполняться, решает контроллер, а не вызывающий.
- **Неудавшееся задание остаётся.** Его запись и журнал сохраняются, чтобы в них можно было посмотреть. Успешное задание очередь покидает. Именно поэтому команда `SHOW` без квалификаторов полезна: остаётся то, что выполняется, и то, что не заладилось.

1.2 Принятые решения и их причины

Два образа, а не один. `batchd` владеет очередью; `batch` — клиент, не владеющий ничем. Противоположное решение — утилита с признаком `setuid`, правящая общий файл, — отдало бы неприкосновенные свойства очереди в руки каждому вызывающему.

Личность устанавливает ядро. Контроллер берёт личность клиента из `SO_PEERCRED`, а не из содержимого запроса. Поэтому клиент может обмануть в чём угодно, кроме того, кто он есть, и всё существенное проверяется именно по этому.

Один контроллер, одна нить. Очередь не запирается, потому что к ней ничто более не обращается. Это настоящее ограничение на то, что можно добавлять: всё, что блокирует основной цикл, останавливает подсистему целиком, — отчего соединения и обслуживаются без блокировки (раздел 2.4).

Дескриптор постоянного размера, без упаковки. `BATCH$JOB` плоский и постоянного размера, поэтому он пишется в журнал и кладётся в сокет как есть. Плата — перемещение почти пустых записей; выигрыш — отсутствие кода упаковки, в котором можно ошибиться, и журнал, читаемый через `od`. Следствие для совместимости изложено в разделе 4.4.

Значения условия, а не коды ошибок. Каждая рутина возвращает значение условия в раскладке OpenVMS: подсистема, номер сообщения, серьёзность. Младший бит — признак успеха, поэтому исход проверяется независимо от того, какое условие он несёт, а причина отличается макросом `$MATCH_COND`, а не сравнением текстов.

Чужая библиотека. Протоколирование, сообщения, разбор командной строки, примитивы очередей и движок FAO взяты из пакета StarLet `utility_routines`. Подсистема не содержит собственного кода общего назначения, а где библиотека оказывалась негодной, исправление вносилось в библиотеку.

1.3 Чего намеренно не делается

- **Подсистема не снимает задание, которым уже не управляет.** Задание живёт в своём сеансе. Когда контроллер останавливается, выполняющиеся задания продолжают работу; при следующем пуске они помечаются осиротевшими либо возвращаются в очередь, если несли `/RESTART`. Этот квалификатор обещает, что задание будет выполнено, а не что оно будет выполнено однажды.
- **У подсистемы нет файла настроек.** Настройки берутся из командной строки контроллера и из сборки. Нет состояния, которое надо разбирать, и нечему рассогласовываться.
- **Подсистема не управляет пользователями.** У неё нет учётных записей, квот и списков доступа. Задание работает от имени подавшего; доступ к записи решается одним владением.

Глава 2. Устройство

2.1 Два образа

```
batch (утилита)                        batchd (контроллер заданий)
---------------                        ---------------------------
разбирает командную строку             владеет очередью
собирает запрос                        планирует задания
открывает сокет         --- AF_UNIX --> порождает процессы
передаёт, ждёт, выводит <-- ответ -----  собирает их код завершения
                                       пишет журнал очереди
```

`batch` не хранит состояния между вызовами. Всё, что переживает команду, находится в контроллере и в двух файлах, описанных в разделе 4.4.

2.2 Очередь

Очередь есть двусвязный список записей `BATCHD$JOBENT`, размещённый в `__QUEUE` библиотеки. Каждая запись обёртывает один дескриптор `BATCH$JOB`. Запись выделяется, когда задание ставится в очередь, и освобождается, когда покидает её, — поэтому контроллер держит столько, сколько нужно его заданиям, а не таблицу, рассчитанную на худший случай.

Обход идёт по ссылкам напрямую — библиотека предлагает снятие с любого конца, но не итератор, — и два макроса в `batchd.c` говорят об этом прямо:

```
#define $JOBQUE_FIRST()     ((BATCHD$JOBENT *) g_jobque.head)
#define $JOBQUE_NEXT(ent)   ((BATCHD$JOBENT *) (ent)->je_links.right)
```

`BATCH_K_MAXJOB` — потолок, а не ёмкость: он существует затем, чтобы забег подателя не исчерпал памяти, а порождаемое им условие `BATCH$_NOSLOT` полезнее подавшему, чем отказ в выделении памяти.

Состояния, через которые проходит задание, изложены в *Руководстве пользователя*, раздел 1.5. В исходных текстах это константы `JOB$K_*` из `batchdef.h`. Значение нуль не назначено: запись в очереди всегда имеет состояние, а запись без состояния не существует.

2.3 Путь запроса

```
batch                     batchd
-----                     ------
connect ----------------> s_batchd$conn_accept
                            SO_PEERCRED, O_NONBLOCK, постановка в g_connque
передача заголовка -----> s_batchd$conn_read     состояние CONN$K_RDHDR
передача нагрузки ------> s_batchd$conn_read     состояние CONN$K_RDPLD
                          s_batchd$conn_dispatch
                            s_batchd$do_submit / do_delete / do_show / release
                            ответ собирается в памяти
приём ответа <----------- s_batchd$conn_write    состояние CONN$K_WRRSP
                          s_batchd$conn_close
```

Ответ собирается целиком прежде, чем передаётся хоть его часть. Именно это не даёт удерживать очередь открытой на время выгрузки медленному читателю: клиент получает очередь такой, какой она была в момент запроса.

2.4 Основной цикл

Рутина `main` в `batchd.c` есть один цикл, и всё, что делает контроллер, происходит в нём:

```
собрать завершившиеся задания      s_batchd$reap
спланировать то, что можно пустить s_batchd$schedule
записать журнал, если что-то из этого изменило очередь
перемотать журнал контроллера, если он вырос
обойти журналы заданий, раз в час
собрать наблюдаемый набор: слушающий сокет, самоканал, каждое открытое соединение
poll
осушить самоканал
продвинуть каждое готовое соединение на один шаг
принять новое соединение
переспланировать и записать журнал, если запрос изменил очередь
```

Два свойства здесь несущие.

Ничто не блокирует. Каждый дескриптор неблокирующий и продвигается только тогда, когда `poll` объявил его готовым. Клиент, подключившийся и замолчавший, стоит одного дескриптора и снимается через `BATCHD$K_CONNTMO` секунд.

Сигналы приходят через самоканал. Обработчик пишет один байт и возвращается; всё прочее делается в основном контексте, где выбор вызываемых рутин не ограничен. Поэтому `SIGCHLD` не состязается с очередью.

Верхний предел ожидания, `BATCHD$K_POLLMS`, существует ради `/AFTER`: наступление назначенного времени само по себе события не порождает, и цикл обязан проснуться и посмотреть.

Глава 3. Модули

3.1 Карта

Модуль	Назначение
`batchdef.h`	Состояния задания, дескриптор `BATCH$JOB`, ёмкости, объявления рутин общего назначения
`batchmsg.h`	Значения условия подсистемы, проверки исхода
`batchmsg.c`	Таблица сообщений и её регистрация
`batchipc.h`	Протокол между двумя образами
`util.c`	Рутины общего назначения: вывод, время, пути, обмен
`batchd.c`	Контроллер заданий
`batch.c`	Утилита
`batchvers.h.in`	Идентификация, заполняемая CMake

Включение идёт в одну сторону: `batchdef.h` тянет `batchvers.h`, `batchmsg.h` и `starlet.h`, а `batchmsg.h` тянет `utility_routines.h`. Ни `batchd.c`, ни `batch.c` никуда не включаются; общего у них — только `util.c` и заголовки.

В одном месте порядок существен. `utility_routines.h` объявляет коды серьёзности перечислением, а `starlet.h` несёт запасные макросы тех же имён под `#ifndef`. Поэтому библиотека должна включаться первой, что `batchdef.h` и обеспечивает. Обратный порядок превращает перечисление в присваивание константе.

3.2 batchdef.h

Определения, общие для оба образов. Три вещи живут здесь и только здесь:

- ёмкости — `BATCH_K_MAXJOB`, `BATCH_K_NPARM`, `BATCH_K_PATHSZ` и прочие;
- состояния задания `JOB$K_*` и условия сохранения записи `RETAIN$K_*`;
- `BATCH$JOB` — дескриптор, описанный в разделе 4.1.

Здесь же изложено соглашение о вызовах, которого держится вся подсистема; о нём раздел 5.3.

3.3 batchmsg.h и batchmsg.c

`batchmsg.h` определяет номер подсистемы и по одному значению условия на каждый различимый исход. Номера сообщений распределены по областям с промежутками, чтобы добавление условия не сдвигало уже назначенные номера:

```
  1 ..   9   общие                 50 ..  69   файлы и журнал
 10 ..  29   общего назначения     70 ..  89   обмен
 30 ..  49   очередь и задания     90 .. 109   командная строка
                                  110 .. 129   порождение процессов и контроллер
```

`batchmsg.c` содержит саму таблицу — массив `EMSG_RECORD` — и рутину `msg$init`, передающую её `__util$inimsg`. Оба образа вызывают `msg$init` прежде, чем что-либо выдать. Три условия на таблицу:

- она **не** `const` — библиотека сортирует её на месте;
- текст сообщения написан для движка **FAO** — `!AZ`, `!SL`, `!UL`, `!SQ`, — поскольку `__util$putmsg_fao` разворачивает его через `__util$faol`. Преобразование printf движок не знает и печатает как есть. Число директив в тексте обязано равняться числу доводов на каждом месте вызова; транслятор этого не проверяет, формат берётся из таблицы во время исполнения;
- конверта `%FAC-S-IDENT` в тексте **нет**: библиотека берёт его из поля `facname` описателя, и текст со своим конвертом оказался бы обрамлён дважды.

3.4 batchipc.h

Протокол. Кадр есть заголовок `BATCH$IPCHDR` постоянного размера, за которым следует `mh_len` байт полезной нагрузки. Порядок байтов не преобразуется: оба конца находятся на одном узле и происходят из одной сборки. Коды функций — `IPC$K_*`, признаки выборки команды `SHOW` — `SHOW$M_*`.

3.5 util.c

Рутины, которыми пользуются оба образа. Ни одна из них не знает ничего о заданиях.

Рутина	Назначение
`util$put`	Собирает строку движком FAO и пишет её в стандартный вывод. Через неё проходит каждая строка, которую утилита показывает пользователю.
`util$mkfspec`	Собирает спецификацию файла движком FAO, отказываясь усечь. Параметры передаются массивом, поскольку переменный список нельзя переслать `__util$faol`.
`util$fmt_time`	Выводит время как `dd-MMM-yyyy hh:mm:ss` через `__util$asctim`.
`util$parse_time`	Разбирает грамматику `/AFTER`. Синтаксический переходник над `__util$bintim`: переписывает спецификацию в форму, принимаемую библиотекой, и оставляет ей арифметику.
`util$parse_cpulist`, `util$fmt_cpulist`	Список `/CPU` в записи `taskset(1)`.
`util$mkpath`	`mkdir -p`.
`util$state_name`	Мнемоника состояния задания.
`util$read_exact`, `util$write_exact`	Блокирующая передача точного числа байтов. Применяются утилитой и для журнала — **не** контроллером для клиентских соединений, которые блокировать нельзя.

3.6 batchd.c

Контроллер. Его рутины делятся на пять групп.

Очередь. `s_batchd$job_alloc` и `s_batchd$job_free` — единственные рутины, создающие и уничтожающие записи; `job_free` вместе с записью удаляет и журнал задания, если не был указан `/KEEP`. `s_batchd$job_by_entry` и `s_batchd$job_by_pid` находят запись; `s_batchd$count_executing` считает занятые места.

Журнал. `s_batchd$journal_save` пишет всю очередь во временный файл и переименовывает его поверх прежней копии, чем достигается неделимость записи. `s_batchd$journal_load` читает журнал обратно и решает, что делать с заданиями, которые выполнялись при остановке контроллера.

Задания. `s_batchd$job_start` порождает процесс: сеанс, сигналы, привязка к процессорам, личность, рабочий каталог, дескрипторы, окружение, `execv`. `s_batchd$schedule` выбирает, что пустить, и освобождает то, что дождалось срока. `s_batchd$reap` собирает код завершения и решает, останется ли запись.

Окружение задания — окружение контроллера, унаследованное через `fork` и `execv`; поверх него `setenv` ставит `BATCH_ENTRY`, `BATCH_NAME`, `BATCH_QUEUE`, `BATCH_RESTART`, `BATCH_RESTARTS` и `P1` … `P8`. Ничего не очищается, `HOME`, `USER`, `LOGNAME`, `SHELL` и `PATH` не ставятся, так что под systemd задание видит `PATH` unit-а. Окружение подавшего до контроллера не доходит: в запросе переменных нет. Стоящие в очереди сценарии полагаются на унаследованный `PATH`; очистка окружения здесь была бы изменением, заметным пользователям.

Запросы. `s_batchd$do_submit`, `s_batchd$do_delete`, `s_batchd$do_show` и ветвь `IPC$K_RELEASE` в `s_batchd$conn_dispatch`. Рутина `s_batchd$job_selected` формулирует правило отбора для `SHOW` в одном месте, поскольку очередь обходится дважды — сосчитать и передать, — и оба прохода обязаны совпадать.

Соединения. `s_batchd$conn_accept`, `conn_read`, `conn_dispatch`, `conn_write`, `conn_close` и состояние, которое они держат в `BATCHD$CONN`. `s_batchd$listen` создаёт слушающий сокет, `s_batchd$log_sweep` очищает каталог журналов от того, что не принадлежит ни одной записи.

Единственная рутина, вызываемая не из основного контекста, — `s_batchd$sighandler`: она пишет байт в самоканал и возвращается, и не делает более ничего.

Порождённый процесс — иной мир. Всё, что следует за `fork` в `s_batchd$job_start`, работает, когда стандартные потоки вот-вот станут журналом задания, поэтому там нельзя выдать сообщение: оно попало бы в журнал задания. Отказ там завершает процесс с кодом `BATCHD$K_EXECFAIL`, который сборщик получает как обычный код завершения.

3.7 batch.c

Утилита. Она разбирает, передаёт и выводит.

Решение содержится в разборе. В DCL квалификатор начинается с косой черты; в Unix с неё же начинается абсолютный путь. Средства CLI библиотеки такого различия не делают и, более того, берут объявленный параметр с головы вектора безусловно. Поэтому здесь делаются две вещи:

- `s_batch$is_qual` классифицирует довод. Он квалификатор лишь тогда, когда текст до знака равенства не несёт своей косой черты и однозначно называет один из квалификаторов таблицы.
- `main` перестраивает вектор доводов — глагол, затем не-квалификаторы, затем квалификаторы — прежде чем отдать его `__cli$parse`. Именно это сохраняет свободу порядка, принятую в DCL, которой средства не предлагают.

Ни один глагол не объявляет параметра, и это намеренно: с объявленным параметром команда `batch submit /NAME=x` отдала бы `/NAME=x` в P1. Единственный параметр каждой команды выбирается из уже расклассифицированных доводов.

`SET ENTRY` — единственный глагол из двух слов. Средства CLI ведут таблицу подглаголов — `ENTRY` есть `CLI_VERB` под `SET`, с таблицей квалификаторов команды — и ждут второе слово сразу за первым, поэтому `main` возвращает первый довод, не являющийся квалификатором, на это место, прежде чем отдать вектор. Диспетчер устанавливает подглагол так же, как глагол: по адресу его дескриптора. `/HOLD`, `/KEEP`, `/RESTART` и `/AFTER` ради него допускают отрицание; `CLI$GET_VALUE` сообщает об отрицённом квалификаторе как о присутствующем, поэтому эти четыре читаются через `CLI$PRESENT`.

Рутина `s_batch$get_quals` извлекает значения из разобранной командной строки. Там остаётся лишь то, что свойственно этой команде: разбор `/PARAMETERS`, разворачивание `/LOG_FILE`, преобразование `/AFTER`. Сопоставление, сокращения, обнаружение неоднозначности и проверка типа принадлежат библиотеке.

3.8 Вендоренный подпроект

UTILITY$ROUTINES устанавливается как пакет `StarLet` и требуется именно им; копии его в этом дереве нет. На одной копии в системе стоит настаивать: раскладка структур той библиотеки объявлена в её заголовках, а подсистема их вкомпилирует — таблицы квалификаторов `batch.c` суть `CLI_PQDESC`, а таблица сообщений `batchmsg.c` — `EMSG_RECORD`, — поэтому вторая редакция в одном образе о себе не заявит.

Что из него используется:

Откуда	Что
`utility_routines.h/.c`	`$PUTMSG_FAO` и реестр сообщений, `__util$deflog`, `__util$rewindlogfile`, примитивы `__QUEUE`
`starlet.h/.c`	Движок FAO, `__util$asctim`, `__util$bintim`, `__util$gettim`, `__util$add_times`
`cli_routines.h/.c`	`__cli$parse`, `__cli$get_value` и таблицы квалификаторов

Собирается он с ключом `-w`: исправлять его не наше дело, а его предупреждение было бы не отличить от нашего.

Когда приходит новая редакция, заменяйте каталог целиком и прогоняйте тесты. Модуль `test/queue.c` существует именно для этого: дефект в `$REMQENT`, однажды исправленный, может вернуться со следующей редакцией, а очередь заданий на нём стоит. Он оправдал себя три редакции подряд — каждая приходила с этим дефектом обратно; с редакции `V.01-09` исправление принято в библиотеку, и каталог вендорится без единой правки.

За одной вещью стоит присмотреть после смены редакции. Библиотека переводит свои имена к общей приставке `__`, модуль за модулем: в одной редакции службы STARLET, в следующей — службы CLI, — поэтому отказ компоновщика с упоминанием `cli$parse` или `util$asctim` означает приставку, а не отсутствующую рутину.

Наши исправления дважды приходилось переносить из редакции в редакцию; теперь они приняты в библиотеку, и каталог вендорится без правок. Оба стоит запомнить как образец: ни одно не проявлялось ошибкой трансляции. В одном значение передавалось адресом там, где принимающая сторона перешла на приём по значению; в другом возвращался статус, серьёзность которого подменялась нечётным числом, а по соглашению `1 & status` нечётное значит успех. С этой редакции библиотека несёт собственные регрессионные проверки, и они прогоняются вместе с нашими.

Глава 4. Структуры данных

4.1 BATCH$JOB

Дескриптор задания, в `batchdef.h`. Плоский, постоянного размера, поля сгруппированы по типу, чтобы не возникало дыр выравнивания. В журнал и в сокет он попадает точно в том виде, в каком лежит в памяти.

Поле	Назначение
`jb_entry`	Номер записи очереди, единственный и возрастающий
`jb_state`	Одно из `JOB$K_*`
`jb_prio`, `jb_retain`, `jb_restarts`	`/PRIORITY`, `/RETAIN`, счётчик перезапусков
`jb_pid`, `jb_uid`, `jb_gid`	Процесс и личность, от имени которой он работает
`jb_exit`, `jb_signal`	Как задание завершилось
`jb_keeplog`, `jb_restart`	`/KEEP`, `/RESTART`
`jb_qtime`, `jb_after`, `jb_stime`, `jb_etime`	Поставлено, не раньше, начато, окончено
`jb_cpus`, `jb_queue`, `jb_name`	`/CPU`, `/QUEUE`, `/NAME`
`jb_script`, `jb_logfile`, `jb_cwd`	Абсолютные пути
`jb_parm`	`/PARAMETERS`, восемь штук

Поля, которые клиенту доверить нельзя — `jb_entry`, `jb_uid`, `jb_gid`, `jb_pid`, отметки времени, — заполняются рутиной `s_batchd$do_submit` и только ею, что бы клиент ни прислал.

4.2 BATCHD$JOBENT и BATCHD$CONN

```
typedef struct batchd_jobent_t {
    ENTRY       je_links;   /* первым: этого требуют макросы $INSQ и $REMQ */
    BATCH$JOB   je_job;
} BATCHD$JOBENT;
```

Ссылки лежат в обёртке, а не в дескрипторе. Именно это делает формат журнала независимым от них: пишется `je_job`, и очередь можно заменить хоть завтра, не касаясь файла.

`BATCHD$CONN` держит то, что соединению нужно между шагами передачи: заголовок запроса, буфер, сколько из него уже перенесено, состояние, личность отправителя и срок. Структура существует потому, что передача не есть один блокирующий вызов.

4.3 Протокол

BATCH$IPCHDR — сигнатура, функция, значение условия в ответе, длина нагрузки, число записей, номер записи — и следом сама нагрузка. SUBMIT несёт BATCH$JOB; SHOW несёт BATCH$SHOWREQ; DELETE и RELEASE не несут ничего, номер записи есть поле заголовка; SET ENTRY несёт BATCH$SETENT, маска которого называет устанавливаемые атрибуты, так что не названный атрибут не трогается.

Ответ на SHOW — заголовок и следом mh_count дескрипторов. Всякий иной ответ — один заголовок.

Значение условия, выработанное контроллером, доходит до утилиты в поле mh_sts. *Доводы* его сообщения не доходят, поэтому утилита сообщает значение, а читаемый текст остаётся в журнале контроллера, где он был выдан полностью.

4.4 Журнал очереди

`BATCHD$JRNHDR` — сигнатура, размер записи, число записей, сохранённый счётчик номеров — и следом столько записей `BATCH$JOB`.

Запись — это **не** образ `BATCH$JOB`. Это последовательность помеченных значений, записанная рутинами TLV библиотеки, а заголовок сообщает формат журнала, а не размер дескриптора. Читающий берёт метки, которые знает, проходит мимо незнакомых и оставляет при значении по умолчанию поле, метки которого нет вовсе. Поэтому журнал переходит через смену редакции в обе стороны: новый BATCH читает старый журнал, а старый BATCH читает новый настолько, насколько его понимает.

Метки и три правила, которыми это свойство держится — метка не переназначается, не меняет типа, а новое поле обязано иметь значение по умолчанию, при котором старый журнал по-прежнему описывает осмысленное задание, — собраны в `batchjrn.h`. Добавить поле в `BATCH$JOB` теперь значит добавить рядом метку, и ничего больше.

Запись идёт в `journal.tmp` и переименовывается поверх `journal.dat`. Переименование неделимо в пределах одной файловой системы, поэтому журнал в недописанном виде не наблюдается. Переименованию предшествует `fsync`, чем и обеспечивается настоящая долговечность и чем запись дорога: см. Приложение А.

Глава 5. Соглашения

Исходные тексты повсюду держатся стиля OpenVMS. Это не украшение: читатель, знающий соглашение, по одному виду имени определяет его время жизни и происхождение.

5.1 Имена

Вид	Смысл
`g_имя`	Глобальное
`l_имя`	Локальное
`s_имя`	Статическое — переменная либо рутина, не вывозимая из своего модуля
`a_имя`	Параметр рутины
`BATCH_K_*`	Константа
`BATCH_M_*`	Признак сборки
`BATCH$_*`	Значение условия
`$МАКРОС`	Макрос
`s_batchd$имя`	Статическая рутина: префикс `s_`, затем модуль, затем имя

Структура объявляется так, что тег несёт суффикс `_t`, а имя типа — разделитель:

```
typedef struct batch_job_t { ... } BATCH$JOB;
```

5.2 Оформление

Whitesmiths: скобка вложенного блока стоит на отступе того тела, которое она открывает, скобка рутины — в нулевой колонке. Только табуляция, шаг восемь; текст до 132-й колонки. Объявления в начале рутины, без отступа, сгруппированы по типу. Одиночное выражение под `if` скобок не получает. Счётчик цикла объявляется в самом `for`, где это возможно.

Каждый модуль открывается полями `MODULE`, `IDENT`, `REV`, `FACILITY`, `ABSTRACT`, `AUTHOR`, `CREATION DATE` и историей ревизий, а каждая рутина — блоком `FUNCTIONAL DESCRIPTION`, где сказано, что она делает, каковы её входные и выходные параметры и что она возвращает.

`IDENT` и `REV` встречаются в модуле дважды — в шапке и в макросах `__IDENT__` и `__REV__` — и обязаны совпадать.

5.3 Соглашение о вызовах

Рутина возвращает значение условия, а результат выдаёт через выходной параметр.

```
if ( $IS_FAILURE(s_batchd$job_by_entry(a_entry, &l_ent)) )
    return  $PUTMSG_FAO(BATCH$_NOSUCHENT, a_entry);
```

Исключений три, и каждое оговорено в своём комментарии: `s_batchd$sighandler`, чья подпись навязана системой, и две рутины `main`.

Макрос `$PUTMSG_FAO` возвращает переданное ему условие — благодаря этому диагностика и возврат укладываются в одно выражение.

Успех проверяется макросом `$IS_SUCCESS`; определённая причина отличается макросом `$MATCH_COND`, который сравнивает опознаватель и не смотрит на серьёзность.

5.4 Диагностика

Вся диагностика идёт через `$PUTMSG_FAO`. Текст находится в таблице, а не в месте вызова; где одно условие обслуживает несколько мест, различающая часть становится параметром текста, а не отдельным условием.

Отказ рутины библиотеки времени исполнения сообщается там, где он произошёл, с именем рутины, её итогом и значением `errno`:

```
if ( 0 > (l_rc = mkdir(l_work, a_mode)) && (errno != EEXIST) )
    return  $PUTMSG_FAO(BATCH$_MKDIRERR, l_work, l_rc, errno, strerror(errno));
```

Вернуть условие наверх и оставить позади рассказ библиотеки о причине отказа — значит потерять единственное, что позволило бы этот отказ опознать.

5.5 Форматирование вывода

Ничто в подсистеме не вызывает `printf` ни в одном из его видов, и ни один модуль не включает `<stdio.h>`. Вывод идёт через движок FAO — `util$put` для строки пользователю, `util$mkfspec` для строки в памяти — директивами OpenVMS:

Директива	Действие
`!AZ`	Строка, завершённая нулём
`!AC`	Счётная строка, контейнер `ASC`
`!UL`, `!SL`	Десятичное без знака и со знаком
`!nZL`	Десятичное с ведущими нулями, шириной n
`!n<...!>`	Поле постоянной ширины, добивка пробелами, усечение
`!n*c`	Знак, повторённый n раз
`!%S`	Буква `s`, если последнее преобразованное число не равно единице

Директива `!/` не применяется. Она выдаёт пару «возврат каретки, перевод строки», принятую в OpenVMS, чего поток Unix не ожидает; разделитель строк пишется литералом `\n`.

Глава 6. Внесение изменений

6.1 Добавление квалификатора

Пусть добавляется квалификатор `/NICE=n` для утилиты.

1. `batchdef.h` — добавьте поле в `BATCH$JOB`, в группу его типа. **Это изменит размер дескриптора**, поэтому существующий журнал будет отвергнут; так и ожидается, иных действий не требуется.
2. `batch.c` — добавьте перечислитель в `QUAL$K_*` и строку в `s_quals`, с типом, который средства CLI должны проверять: `CLI$K_NUM`, `CLI$K_QSTRING`, `CLI$K_OPT` для квалификатора без значения, `CLI$K_KWD` со списком `CLI_KEYWORD` для набора слов.
3. `batch.c` — добавьте поле в `BATCH$CLICTX`, извлеките значение в `s_batch$get_quals`, положите его в дескриптор в `s_batch$do_submit`.
4. `batch.c` — впишите квалификатор в текст подсказки `s_batch$usage` и в раздел `ABSTRACT` модуля.
5. `batchd.c` — примените его, обычно в `s_batchd$job_start`.
6. `batch.c` — выведите его в `s_batch$show_full`, если он стоит того, чтобы его видеть.
7. *Руководство пользователя* — описание квалификатора в словаре команд.
8. `test/smoke.sh` — проверка, что квалификатор принимается, а негодное значение отвергается.

Следите за сокращениями: новый квалификатор, совпадающий началом с существующим, удлиняет кратчайшее однозначное сокращение обоих, и это видно пользователю.

6.2 Добавление условия

1. `batchmsg.h` — возьмите следующий свободный номер **в нужной области** и дайте условию ту серьёзность, которая описывает исход, а не настроение вызывающего.
2. `batchmsg.c` — добавьте строку в `s_msgtab`, директивами FAO и без собственного конверта. Число директив в тексте обязано равняться числу доводов, передаваемых на каждом месте вызова; транслятор этого не проверяет, поскольку формат берётся из таблицы во время исполнения.
3. *Руководство пользователя*, Глава 6 — мнемоника, пояснение и требуемое действие.

Проверка, которую стоит прогонять после правки таблицы:

```
# каждый текст и справочник сообщений в руководствах несут директивы FAO, а не преобразования
# printf, и вся диагностика идёт через $PUTMSG_FAO
```

Эту проверку делает `test/smoke.sh` (SMOKE.SH X-01-26); она написана после того, как текст `IVENTRY` дошёл до сборки с `%s`, который движок FAO печатал как есть. Число доводов вызова против директив его текста дерево по-прежнему не проверяет: считайте их вручную.

6.3 Добавление команды

1. `batchipc.h` — код функции `IPC$K_*` и структура нагрузки, если команда её несёт.
2. `batch.c` — глагол в `s_verbs`, обработчик рядом с `s_batch$do_submit` и ветвь в `main`.
3. `batchd.c` — `case` в `s_batchd$conn_dispatch` и обработчик рядом с `s_batchd$do_submit`. **Собирайте ответ в памяти**; из обработчика в дескриптор не пишите.
4. Установите `*a_dirty`, если команда изменила очередь, чтобы цикл переспланировал её и записал журнал.
5. *Руководство пользователя* — описание в словаре команд.

6.4 Изменение дескриптора задания

Любое изменение `BATCH$JOB` меняет `sizeof` и тем отвергает существующий журнал. Это и есть задуманный механизм, отдельный номер версии ему не нужен. Нужна ему запись в истории ревизий `batchdef.h`, где об этом сказано: читатель системы, потерявшей очередь после обновления, вправе найти причину.

Два неприкосновенных свойства должны сохраниться:

- дескриптор остаётся плоским и постоянного размера — никаких указателей, ничего, что имеет смысл только в том адресном пространстве, которое это записало;
- поля, которые клиенту устанавливать не дозволено, устанавливаются только рутиной `s_batchd$do_submit`.

6.5 Добавление проверки в тест

Сценарий `test/smoke.sh` поднимает контроллер на временном корневом каталоге и работает с ним через утилиту. Проверка состоит из трёх частей: действие, ожидание нужного состояния и `l_check`.

```
"${g_batch}" submit "${g_root}/scripts/ok.sh" /NAME=probe >/dev/null 2>&1
l_wait_for "probe" show /all
l_check $? "the job reached the queue"
```

Ждите состояния, а не отсчитывайте время. Сколько занимает задание, зависит от загрузки машины и заранее не известно. Рутины `l_wait_for` и `l_wait_gone` опрашивают состояние в течение десяти секунд.

Тест прибирает за собой: порождённые им задания снимаются по группе процессов, поскольку задание переживает контроллер, а часть заданий проверка перезапуска намеренно оставляет осиротевшими.

Глава 7. Система сборки

7.1 Состав

Файл	Назначение
`CMakeLists.txt`	Идентификация, общие ключи трансляции, порождаемый заголовок, вендоренный подпроект, цель удаления
`src/CMakeLists.txt`	Цели сборки и установка
`test/CMakeLists.txt`	Регистрация тестов
`etc/batchd.service.in`	Шаблон unit-файла systemd
`etc/batchd.init.in`	Шаблон сценария запуска System V, для системы без systemd
`cmake/uninstall.cmake.in`	Сценарий удаления
`doc/mkpdf.py`	Вёрстка руководств в PDF
`src/batchvers.h.in`	Шаблон заголовка идентификации

`batchcom` — объектная библиотека, содержащая `util.c` и `batchmsg.c`; её линкуют оба образа, поэтому ни один из модулей не транслируется дважды и оба образа собраны из одного и того же объектного файла.

Особняком от образов стоит одна цель: `doc_pdf` заново верстает руководства из их markdown и собирается вместе со всем прочим, когда установлен `BATCH_M_PDF`. Пишет она в `doc/`, рядом с markdown, а не в дерево сборки: свёрстанные руководства принадлежат исходным текстам и хранятся вместе с ними под управлением версиями — чтобы чтение руководства не требовало установленного TeX, а подсистема передавалась одним набором. Устанавливайте `BATCH_M_PDF` после правки руководства и фиксируйте свёрстанное вместе с правкой.

7.2 Ключи трансляции

Они собраны в интерфейсной цели `batch_options`, а не в глобальном `CMAKE_C_FLAGS`, чтобы можно было добавить цель, не повторяя их:

```
-Wall -Wextra -Wno-unused-parameter -Wformat=2 -Wshadow -Wpointer-arith -Wwrite-strings
```

`BATCH_M_STRICT=ON` добавляет `-Werror`. **Вендоренный подпроект собирается с `-w`** и от всего этого избавлен.

Стандарт — gnu11: знак доллара в идентификаторе есть расширение GNU, и на нём стоит всё соглашение об именах.

7.3 Одна неловкость

`configure_file` подставляет `@ИМЯ@`, но не распознаёт знак доллара внутри имени переменной. Поэтому настройки доходят до шаблонов через вспомогательные имена без разделителя — `BATCH_CFG_SPOOLROOT` несёт значение `BATCH_K_SPOOLROOT`, — а порождаемые из них макросы разделитель возвращают.

Глава 8. Тесты

8.1 Что есть

Цель	Что проверяет
`status_convention`, `cli_features`, `tlv`	Регрессионные проверки пакета StarLet; прогоняются вместе с нашими и подчинены тому же выключателю
`batch_queue`	Примитивы очередей этой библиотеки: снятие головы, хвоста, середины и последней записи, с обходом ссылок в обе стороны после каждого снятия
`batch_smoke`	Подсистему целиком, через утилиту, на временном корневом каталоге

```
ctest --test-dir build --output-on-failure
```

Тест считается провалившимся, когда его вывод содержит `%BATCH-[EF]-`, поэтому проверка сообщает свой исход сама, а `ctest` о ней ничего знать не должен.

8.2 Что стережёт основной тест

Каждая проверка существует потому, что что-то однажды было неверно либо могло незаметно испортиться:

Проверка	Что стережёт
Задание сохранено при неудаче, снято при успехе	Правило сохранения записи, которым и полезна команда `SHOW`
`/HOLD` и `/AFTER`	Состояние удержания и освобождение из него
Предел на число заданий	Планировщик
`DELETE`	Снятие записи и сигнал выполняющемуся заданию
Перезапуск контроллера	Журнал очереди
Абсолютный путь как параметр	Различение квалификатора и пути
Формы `+3`, `+:30`, `+::45`	Дефект разбора, до которого единственная проверявшаяся форма не доходила
Условия с мнемониками	Средства сообщений
`/RESTART`	Возврат в очередь против осиротения
Молчащий клиент	То, что соединения обслуживаются без блокировки
Уборка журналов и `/CPU`	Уборку и проверку списка процессоров
Задание, ничего не запускающее, переживает перезапуск	Признак изменения, который отбрасывался

Две последние строки стоит прочесть дважды: обе проверки написаны **после** того, как дефект дошёл до сборки, а иначе и не узнать, нужна ли проверка.

Приложение A. Известные ограничения

Каждое из них есть решение, которое можно пересмотреть, и каждое указано вместе с местом, где живёт.

Журнал очереди пишется целиком и сбрасывается на носитель при каждом изменении. `s_batchd$journal_save`. Измерено на очереди из 300 записей на ext4: 4579 мкс на операцию, записывающую журнал, против 29 мкс на операцию, которая его не пишет, из чего на сброс приходится около 3700 мкс. Три пути, по возрастанию объёма работ: объединять записи в пределах короткого промежутка; сделать сброс предметом настройки; писать журнал приращениями и уплотнять его время от времени.

Одна очередь. Дескриптор несёт поле `jb_queue`, и команда `SHOW` по нему отбирает, но предел на число заданий общий для всей подсистемы. Вторая очередь означала бы предел на каждую очередь и планировщик, который о них знает: `s_batchd$schedule`.

Дескриптор почти пуст. 3752 байта, из которых `jb_parm` занимает 2048. И журнал, и ответ на `SHOW` несут его целиком. Сокращение области параметров есть видимое пользователю сужение; упаковка при передаче стоила бы того свойства, что кода упаковки нет вовсе.

Значение nice, ограничения ресурсов и управляющая группа для отдельного задания не задаются. Привязка к процессорам задаётся, начиная с `/CPU`; остальное наследуется от контроллера. Место для любого из них — `s_batchd$job_start`.

`/JOB_LIMIT` по умолчанию равен 4 независимо от машины. Теперь, когда контроллер при запуске читает свою маску привязки, число процессоров в ней есть очевидное значение по умолчанию. Рутина `main` в `batchd.c`.

Контроллер просыпается раз в секунду, даже когда бездействует. `BATCHD$K_POLLMS`, ради `/AFTER`. Вычисление ближайшего срока убрало бы пробуждения, не меняя поведения.

Диагностика разбора командной строки выходит под именем `%CLI_RTNS-`, а не `%BATCH-`, поскольку библиотека несёт своё имя подсистемы. Оставлено намеренно: читатель отличает замечание о форме команды от замечания о её смысле.